

Lesson: Data Cleaning and Handling Missing Values

Before any machine learning model can perform well, the **quality of the data** it's trained on must be high. Real-world data is often messy, incomplete, inconsistent, or full of errors — and feeding this raw data into an ML model can lead to poor results.

In this lesson, we'll explore one of the most critical steps in the machine learning pipeline: **data cleaning**. You'll learn how to identify and handle **missing values**, detect anomalies, and prepare your dataset for reliable analysis.

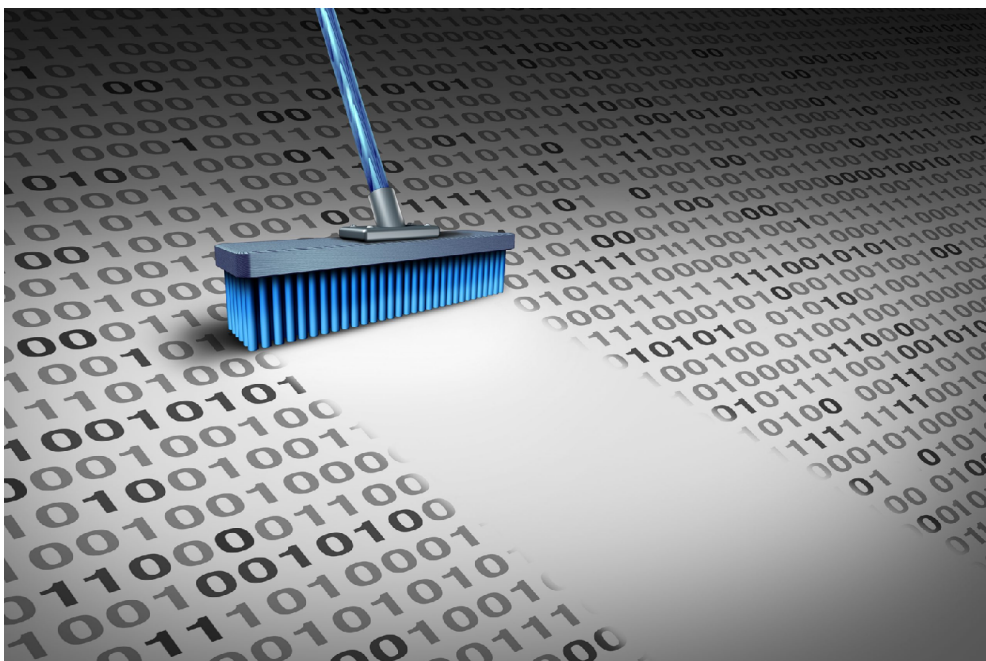
Learning Outcomes

By the end of this lesson, students will be able to:

- Handle missing values using appropriate techniques
 - Detect and manage outliers with statistical methods
 - Encode categorical variables using label and one-hot encoding
 - Understand the importance of train/test splits to prevent data leakage
-

Data Cleaning: Preparing Reliable Data for Analysis

Data cleaning is the process of detecting and correcting (or removing) inaccurate, inconsistent, incomplete, or improperly formatted data from a dataset. This step is essential for ensuring that your analysis or machine learning models produce **valid and reliable results**.



Issues such as duplicates, missing values, or mislabeled entries are common—especially when data comes from multiple sources. Even small data quality problems can lead to **misleading outcomes**,

making models appear accurate when they're actually built on faulty foundations.

While there's no universal set of steps for cleaning all datasets—since the process often depends on the specific structure and content of the data—it's important to follow a **systematic, repeatable approach**. Establishing a clear template for your data cleaning process ensures **consistency and quality** in your results every time.

Data Quality Characteristics

Effective data cleaning aims to produce high-quality data that supports accurate, trustworthy analysis. But what defines *quality* in data? The following characteristics serve as key indicators:



- **Validity:** Data should conform to defined rules, formats, or business constraints. For example, dates must follow the correct format, and values should fall within acceptable ranges.
- **Accuracy:** Data must correctly represent real-world facts or events. Inaccurate data can distort findings and lead to incorrect conclusions.
- **Completeness:** All required information should be present. Missing values—especially in critical fields—can compromise analysis and model performance.
- **Consistency:** Data should not conflict across different systems or parts of the dataset. For example, a user's name or ID should match wherever it appears.
- **Timeliness:** The data should be current and reflect the present state of the world. Outdated information may no longer be relevant or useful for decision-making.

Ensuring these qualities helps create a solid foundation for analysis, modeling, and decision-making. Without them, even the most sophisticated models can yield unreliable results.

Examples of Data Quality

Understanding how these characteristics appear in real datasets can help you spot problems more easily. Here are some common issues mapped to the quality dimensions:

Quality Dimension	Example of a Problem	Explanation
Validity	"Email" field contains values like <code>12345</code>	Fails to meet expected format for an email address
Accuracy	A customer's age is listed as <code>250</code>	Clearly incorrect, as it's outside the range of realistic human ages
Completeness	Missing address fields in shipping records	Required data is absent
Consistency	Customer listed as "Active" in one table, "Inactive" in another	Conflicting values across different sources
Timeliness	Dataset includes last year's inventory marked as "current"	The data is outdated and doesn't reflect present conditions

Identifying and Handling Missing Values in Datasets

Missing values are one of the most common issues in real-world datasets. They can occur for many reasons, data entry errors, system malfunctions, or simply because certain information wasn't available at the time of collection.

Handling missing data properly is critical, as ignoring it can lead to biased analyses, reduced statistical power, or inaccurate predictions in machine learning models.

Identifying missing values in datasets

Before you can fix missing data, you need to *detect* it. Missing values are often not immediately obvious, especially when datasets are large or inconsistently formatted.

Common Representations of Missing Data

Missing values can appear in several forms, including:

- Blank or empty cells
- Special strings like `NA`, `NaN`, `null`, or `N/A`
- Placeholder values such as `0`, `-1`, or `-999` (used when data is unavailable)
- Inconsistent formats that fail validation (e.g., `??` in a numeric field)

Recognizing these is the first step in assessing the extent of the problem.

Example: Identifying Missing Values

Python Code for Handling Common Missing Values

```

import pandas as pd

# Define common string and numeric placeholders for missing values
missing_strings = ['NA', 'NaN', 'null', 'N/A', '??']
missing_numbers = [0, -1, -999]

# Read CSV and handle string-based missing values
df = pd.read_csv('your_data.csv', na_values=missing_strings)

# Convert numeric columns explicitly to handle invalid entries like '??' or 'text'
for col in df.select_dtypes(include='object'):
    try:
        df[col] = pd.to_numeric(df[col], errors='coerce')
    except:
        pass # Keep non-numeric columns as-is

# Replace numeric placeholders with NaN in numeric columns only
for col in df.select_dtypes(include=['int64', 'float64']):
    df[col] = df[col].replace(missing_numbers, pd.NA)

# Optional: Check for missing values after cleanup
print("Missing values per column:")
print(df.isnull().sum())

# Optional: Preview cleaned data
print("\nCleaned DataFrame:")
print(df.head())

```

Common Strategies for Handling Missing Data

Once you've identified missing values in your dataset, the next step is to decide how to handle them. The best strategy depends on the context, the type of data, and how much is missing. Below are the most widely used approaches:

1. Deletion (Dropping Missing Data)

When to use: When the missing data is minimal and appears randomly.

a. Drop rows with any missing values:

```
df_cleaned = df.dropna()
```

b. Drop columns with too many missing values:

```
df_cleaned = df.dropna(axis=1, thresh=len(df) * 0.5) # keep columns with
≥50% non-null
```

2. Imputation (Filling Missing Values)

When to use: When removing data would result in too much information loss.

a. Fill with a constant value:

```
df['category'].fillna('Unknown', inplace=True)
```

b. Fill numeric columns with mean, median, or mode:

```
df['age'].fillna(df['age'].mean(), inplace=True)
df['income'].fillna(df['income'].median(), inplace=True)
df['city'].fillna(df['city'].mode()[0], inplace=True)
```

3. Forward or Backward Fill (Time Series or Sequential Data)

When to use: For sequential data where nearby values are likely similar.

```
df.fillna(method='ffill', inplace=True) # forward fill
df.fillna(method='bfill', inplace=True) # backward fill
```

4. Predictive Imputation (Advanced)

When to use: When you want to estimate missing values using a machine learning model based on other features.

```
from sklearn.impute import KNNImputer

imputer = KNNImputer(n_neighbors=3)
df_imputed = pd.DataFrame(imputer.fit_transform(df), columns=df.columns)
```

This is more complex and works best when you have a lot of features and enough data to train an imputation model.

Types of missing data

Not all missing data is the same. Understanding *why* data is missing helps determine the best way to handle it. Statisticians classify missing data into three types:



Missing completely at random (MCAR):

The missingness is completely unrelated to both observed and unobserved values in the dataset. In simpler terms, the absence of data occurs randomly, without any discernible pattern. MCAR data can be handled effectively through simple methods like listwise deletion or mean imputation, without compromising the integrity of the analysis.

A classic example of MCAR is when a survey participant unintentionally skips a question. The likelihood of data being missing is independent of any information present in the dataset. This mechanism is considered the most desirable for data analysis, as it does not introduce any bias.

Missing at random (MAR):

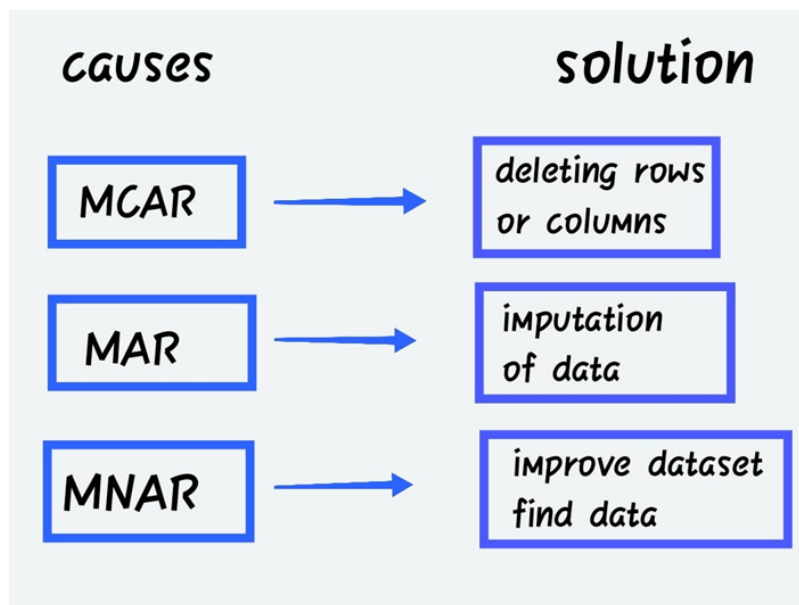
In MAR, the missingness can be explained by some observed features in the dataset. Although the data is missing systematically, it is still considered random because the missingness is not related to the unobserved values. MAR data requires more sophisticated handling techniques like multiple imputation or maximum likelihood estimation.

For instance, in a tobacco study, younger participants might report their values less often (regardless of how much they smoke), leading to systematic missingness for age-related reasons.

Missing not at random (MNAR):

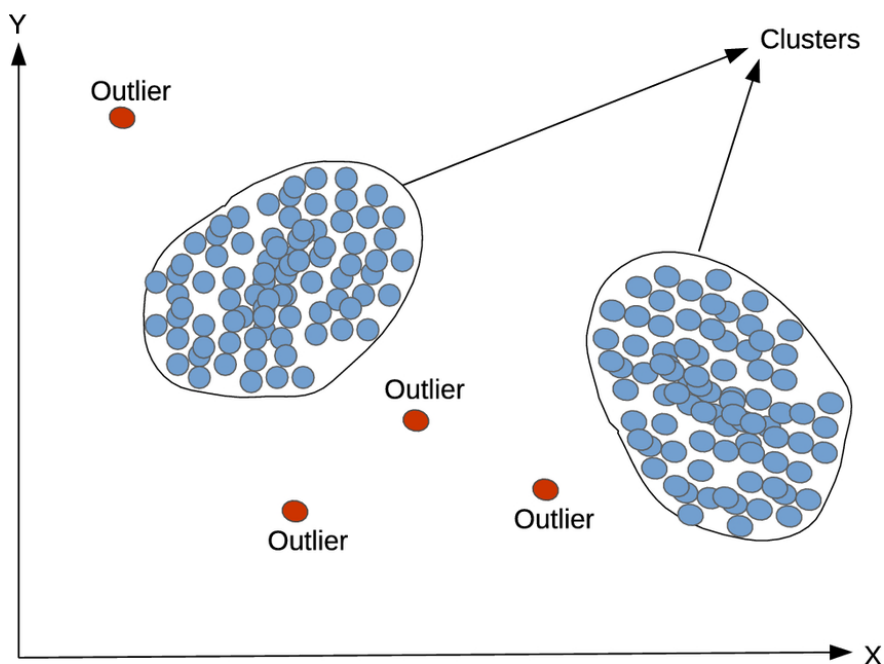
MNAR occurs when the missingness itself is related to the unobserved data. In this case, the missing data is not random and is associated with specific reasons or patterns. MNAR data is the most challenging to handle, as the reasons for missingness are not captured within the observed data.

Referring to the tobacco study example, participants who smoke the most might intentionally withhold their smoking habits, leading to systematic missingness related to the missing data.



Detect and manage outliers using statistical techniques

Outliers are data points that differ significantly from other observations. They can arise due to variability in the data, measurement errors, or data entry issues. Detecting and managing outliers is essential, as they can distort statistical analyses and reduce the accuracy of models.



Types of Outliers:

Extreme Outlier: Data points that lie far from the mean or median, typically beyond 3 times the interquartile range (IQR).

Mild Outlier: Data points that are moderately different from the rest of the data, falling between 1.5 to 3 times the (IQR) from the quartiles.

Detecting Outliers

There are several ways to identify outliers statistically. Here are the most common techniques:

1. Z-score Method:

The **Z-score** measures how many standard deviations a data point is away from the mean. A common threshold for detecting outliers is a Z-score greater than **3** or less than **-3**.

```
from scipy.stats import zscore
import pandas as pd

df = pd.read_csv('your_data.csv')

# Calculate Z-scores for a numeric column
z_scores = zscore(df['income'])

# Identify outliers
outliers = df[abs(z_scores) > 3]
print("Z-Score Outliers:")
print(outliers)
```

2. IQR (Interquartile Range) Method

- This method uses the spread of the middle 50% of the data.
- The Interquartile Range (IQR) is the range between the first quartile (Q1) and the third quartile (Q3). Any data points exceeding 1.5 times the IQR from Q1 or Q3 are considered outliers. This method is helpful if you have a few values on the extreme ends of your dataset, but you aren't sure whether any of them might count as outliers.

Formula: $(IQR = Q3 - Q1)$

Lower Bound:

$$(Q1 - 1.5 * IQR)$$

Upper Bound: $(Q3 + 1.5 * IQR)$

Steps of IQR

- 1.Sort your data from low to high
- 2.Identify the first quartile (Q1), the median, and the third quartile (Q3).
- 3.Calculate your $IQR = Q3 - Q1$
- 4.Calculate your upper fence = $Q3 + (1.5 * IQR)$
- 5.Calculate your lower fence = $Q1 - (1.5 * IQR)$
- 6.Use your fences to highlight any outliers, all values that fall outside your fences.


```

Q1 = df['income'].quantile(0.25)
Q3 = df['income'].quantile(0.75)
IQR = Q3 - Q1

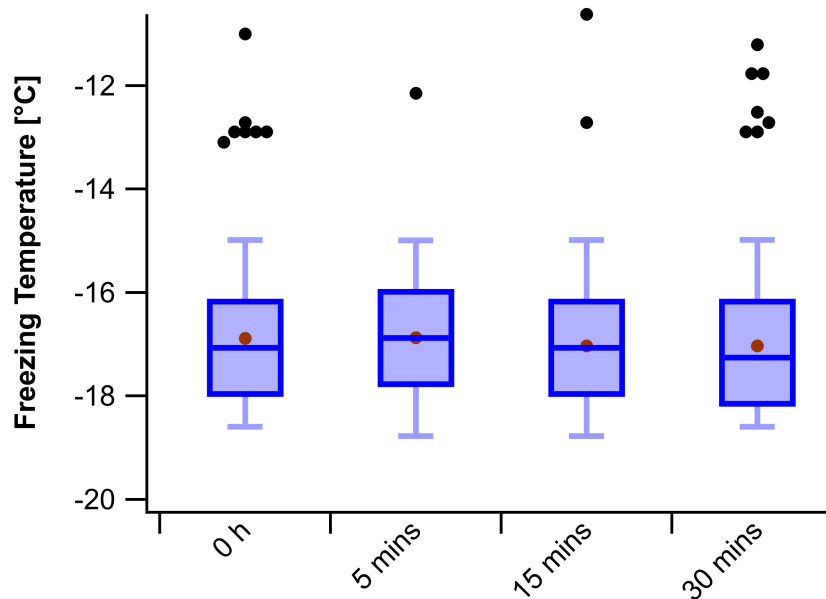
# Define outlier bounds
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# Identify outliers
outliers = df[(df['income'] < lower_bound) | (df['income'] > upper_bound)]
print("IQR Outliers:")
print(outliers)

```

3. Boxplots & visualization techniques:

You can use libraries in Python to visualize your data with a box plot, a box-and-whisker plot, a histogram, and a scatter plot, allowing you to see the data distribution at a glance. These types of charts highlight the minimum and maximum values (the range), the median, and the interquartile range. for your data.



Managing Outliers

Once outliers are detected, managing them effectively is essential to avoid skewing your analysis or degrading machine learning model performance. The best strategy depends on the nature of the data, the model being used, and the cause of the outlier. Below are the most common and effective methods:

A. Remove Outliers

- **When to use:** When outliers are clearly the result of errors or noise (e.g., invalid sensor readings or data entry mistakes).

- **How to do it:** Remove data points that fall outside a defined range (e.g., using Z-score or IQR).

```
# Using IQR method
df_clean = df[(df['column'] >= lower_bound) & (df['column'] <=
upper_bound)]
```

B. Cap or Floor (Winsorizing)

- **When to use:** When you want to retain all rows but limit the influence of extreme values.
- **How to do it:** Replace outliers with the values at a specified percentile (e.g., 5th and 95th).

```
lower_cap = df['column'].quantile(0.05)
upper_cap = df['column'].quantile(0.95)
df['column'] = df['column'].clip(lower=lower_cap, upper=upper_cap)
```

C. Transform Outliers

- **When to use:** When the data is skewed due to outliers, and you're using models that assume normality or linearity.
- **How to do it:**

```
import numpy as np
# Log Transformation
df['log_column'] = np.log1p(df['column'])

# Square Root Transformation
df['sqrt_column'] = np.sqrt(df['column'])

# Box-Cox Transformation (only for positive values)
from scipy import stats
df['boxcox_column'], _ = stats.boxcox(df['column'])
```

D. Impute Outliers

- **When to use:** When you don't want to remove or cap, but still want to soften the impact of an outlier.
- **How to do it:** Replace with statistical values like the median.

```
median = df['column'].median()
df.loc[(df['column'] < lower_bound) | (df['column'] > upper_bound),
'column'] = median
```

E. Bin or Bucket Outliers

- **When to use:** For classification or segmentation tasks, or when individual outlier values don't matter.

```
# Create quartile-based bins
df['binned_column'] = pd.qcut(df['column'], q=4, labels=["Low", "Medium",
"High", "Very High"])
```

F. Keep and Flag

- **When to use:** If outliers are valid and may carry important meaning (e.g., medical anomalies, VIP customers).
- **How to do it:** Keep the data, but flag the row for further analysis or model consideration.

```
df['is_outlier'] = (df['column'] < lower_bound) | (df['column'] >
upper_bound)
```

Best Practices for Managing Outliers

Method	Use Case	Keeps Data	Notes
Remove	Clear errors or invalid values	✗	Fast, but can lead to data loss
Cap/Floor	Limit outlier effect without removal	✓	Good for regression models
Transform	Reduce skew for models that assume normality	✓	Use log/sqrt/Box-Cox based on data
Impute	Extreme but legitimate	✓	Use median for robustness
Bin	Grouping values for analysis or models	✓	Useful for categorical interpretation
Keep & Flag	Outliers are informative or domain-relevant	✓	Best for anomaly detection or audit

Transforming Categorical Data

Categorical data consists of variables that represent distinct groups or categories rather than numerical values. Machine learning models typically require numeric input, so transforming categorical data into a suitable numerical format is a key preprocessing step.

Common Transformation Techniques

1. One-Hot Encoding (OHE):

OHE is a method that creates a binary column for each category to translate categorical variables into numerical values. It is helpful when working with multi-level categorical variables. For example, let's say we have a dataset of tshirts with a column called "color" that contains the following values: "red", "green", and "blue".

Color	Price	Units
Red	300	2
Green	800	3
Blue	200	1
Red	700	1
Green	600	1

One-hot encoding would create three new binary columns, one for each unique category, with a value of 1 indicating that the category is present and a value of 0 indicating that it is not. The resulting data might look like this:

Color	Price	Units	Color_Red	Color_Green	Color_Blue
Red	300	2	1	0	0
Green	800	3	0	1	0
Blue	200	1	0	0	1
Red	700	1	1	0	0
Green	600	1	0	1	0

In Python, You can use the `get_dummies()` function from the pandas library to apply one-hot encoding to the "color" column of your dataframe. Here is an example of how to do it:

```
import pandas as pd

# Create an example dataframe
df = pd.DataFrame({'color': ['red', 'green', 'blue', 'red', 'green'],
                    'price': [500, 800, 300, 400, 600],
                    'units': [2, 3, 1, 1, 1]})

# Apply one-hot encoding to the "color" column
df_encoded = pd.get_dummies(df, columns=['color'])
```

2. Label Encoding:

It is a technique used to convert categorical variables into numerical values by assigning a unique integer value to each category. It is useful for handling ordinal variables, where the order of the categories matters.

For example, let's say we have a dataset with a column called "size" that contains the following values: "small", "medium", "large". Label encoding would replace each category with an integer, such as: "small" = 0, "medium" = 1, "large" = 2. The resulting data might look like this:

Size	Encoded_Size
Small	0

Size	Encoded_Size
Medium	1
Large	2
Small	0
Medium	1

```
from sklearn.preprocessing import LabelEncoder

# Create example dataframe
df = pd.DataFrame({'size': ['small', 'medium', 'large', 'small', 'medium'],
                    'price': [500, 800, 300, 400, 600],
                    'units': [2, 3, 1, 1, 1]})

# Create an instance of the encoder
encoder = LabelEncoder()

# Fit and transform the "size" column
df['encoded_size'] = encoder.fit_transform(df['size'])

print(df)
```

Trade-offs between One-Hot and Label Encoding:

Encoding Method	One-Hot	Label Encoding
Advantages	Prevents the model from assuming any inherent order	Simple and efficient for ordinal data. Does not increase dimensionality.
Disadvantages	Can lead to high dimensionality with many unique categories.	Imposes an arbitrary order on non-ordinal data. Model might assume false relationships

Importance of Train/Test Split & Avoiding Data Leakage

When building machine learning models, one of the most crucial steps is dividing your dataset into separate **training** and **testing** sets. This allows you to evaluate how well your model generalizes to *unseen data*. But more importantly, this step helps prevent a major issue known as **data leakage**.

What Is Data Leakage?

Data leakage occurs when information from outside the training dataset is used to create the model — either directly or indirectly. This results in overly optimistic performance metrics during model evaluation but poor performance in real-world use.

Why split data into train and test sets?

Splitting your data into **training** and **testing** sets helps mimic the real-world scenario where a model is trained on existing data and then used on new, unseen data. Then, the model has:

- Improved model accuracy and efficiency.
- Reduced model overfitting.
- A relatively fast way to evaluate model performance with new data.

Example: split dataset into 80% training and 20% testing

```
from sklearn.model_selection import train_test_split

# Example: Splitting 80% training and 20% testing
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

Common Causes of Data Leakage:

1. Including future information in training data.
2. Improper feature engineering that uses test data insights.
3. Incorrect use of scaling before splitting data.

Preventing Data Leakage in Machine Learning:

Data Preprocessing: Apply preprocessing steps (scaling, encoding, imputation) **separately** for training and test sets.

Proper Data Splitting: Ensure correct **train/test split** to prevent test data from leaking into training. For **time-dependent data**, split chronologically to avoid future data entering the training phase. Use a **separate validation set** to check for leakage and ensure real-world representation.

Cross-Validation: Use **k-fold cross-validation** to evaluate models on multiple data subsets. Helps detect potential leakage and improves generalization.

Feature Engineering: Avoid features that introduce **future data** into training. Regularly assess feature relevance to ensure they only reflect available information at prediction time.

Time-Based Validation (for Time-Series Data): Use **time-based validation** to ensure past data is used to predict future outcomes.

Apply methods like **rolling window validation** or **walk-forward validation** to prevent future data leakage.

Implementing `train_test_split` using Scikit-learn.

```
from sklearn.model_selection import train_test_split

# Sample dataset
df = pd.DataFrame({'Feature': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
                   'Target': [0, 1, 0, 1, 0, 1, 0, 1, 0, 1]})

# Splitting data (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(df[['Feature']],
                                                    df['Target'], test_size=0.2, random_state=42)

print("Training Data:\n", X_train)
print("Test Data:\n", X_test)
```

Conclusion:

This lesson covered essential data cleaning techniques, including handling missing values, detecting and managing outliers, and encoding categorical data. We explored methods like imputation, removal, and forward/backward fill for missing data, as well as Z-score and IQR for outlier detection. Additionally, we highlighted the importance of proper train/test splitting to prevent data leakage. Mastering these techniques ensures cleaner datasets, leading to more reliable and accurate machine learning models.